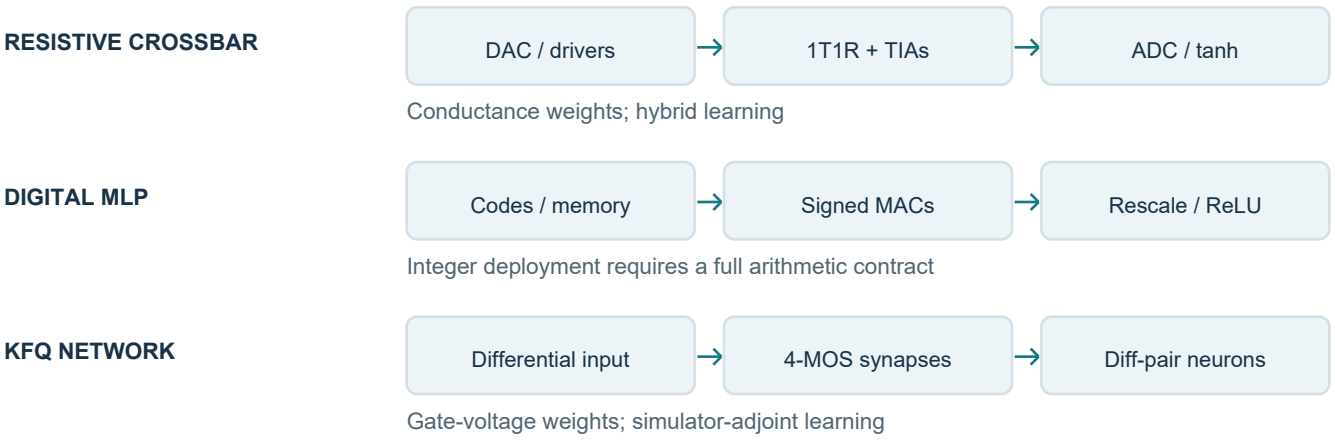
Analog crossbars, digital MLPs, and your Kirchhoff designs

How they compute, learn, and deploy - and how to compare them fairly



A literature-grounded audit of the local resistive reference, digital reference, four-quadrant KFQ, and closed differential-pair ring.

30 selected references. Primary papers, official documentation, and inspected source code. Includes original derivations, implementation findings, historical evidence, and a staged experiment plan.

Prepared 12 October 2026. No new training runs or silicon measurements are claimed.

Contents

Executive assessment 3

1. Research scope and evidence rules 4

2. What distinguishes the architectures 5

3. Analog crossbar operation: the physical computation 6

4. Crossbar interfaces, addressing, and scaling 7

5. Crossbar training: three distinct workflows 8

6. Updates, granularity, and programming error 9

7. Digital MLP operation and an implementable integer path 10

8. Digital QAT and the current implementation gaps 11

9. Digital circuit verification: what is actually available 12

10. KFQ: four-quadrant MOS multiplication 13

11. Differential-pair neurons and operating constraints 14

12. Adjoint training and hardware realizability 15

13. The ring, recurrent-core proposal, and published KirchhoffNet 16

14. The simulator fidelity finding 17

15. Existing results: useful evidence, incompatible experiments 18

16. A fair comparison requires three experiment tracks 19

17. Data, tuning, and statistical testing 20

18. Verification ladder before benchmarking 21

19. Energy, power, and training cost 22

20. Latency, throughput, and area 23

21. Robustness and the experiment matrix 24

22. A concrete implementation and publication roadmap 25

23. Suggested reporting template and decision criteria 26

24. Annotated reading guide 27

References 28

Appendix A. Local evidence map and limits 30

Appendix B. Glossary 31

Executive assessment

Your most credible comparison is between three implementations of a small, biased, nonlinear classifier: a resistive crossbar with measured forward computations, a digital integer MLP with a verified deployment path, and the four-quadrant MOS network in kfabric. The older closed differential-pair ring needs a separate evaluation because it relies on a learned software classifier. The recurrent-core proposal in kfabric's architecture document is also a separate design; it should not be described as the implemented feedforward KFQ network.

The current evidence supports learning capability in the simulated analog circuits. It does not yet support a complete end-to-end energy, latency, or area advantage. Converter costs are missing from the explicit crossbar netlists; weight storage and gate-bias generation are missing from the explicit KFQ netlists; and the digital baseline lacks a complete signed integer inference implementation and a mapped network circuit.

The most consequential discovery is in the simulator itself. Its active Level-1 MOS implementation accepts source-body voltage but does not use it; it sets body transconductance and terminal charges to zero, and does not use the model-card channel-length-modulation parameter. Thus the supplied GAMMA, PHI, and LAMBDA values do not make the present results realistic with respect to those effects. Exact cancellation, extremely small settling times, and tiny ideal-source power must be interpreted within that simplified model.

Recommended claim to pursue: a fully differential CMOS network can implement a useful signed nonlinear classifier while reducing internal conversion boundaries. Establish that claim with matched software controls, robust circuit verification, realistic bias storage and loading, and accuracy-conditioned system metrics. Do not lead with universal superiority over crossbars or digital processors.

1. Research scope and evidence rules

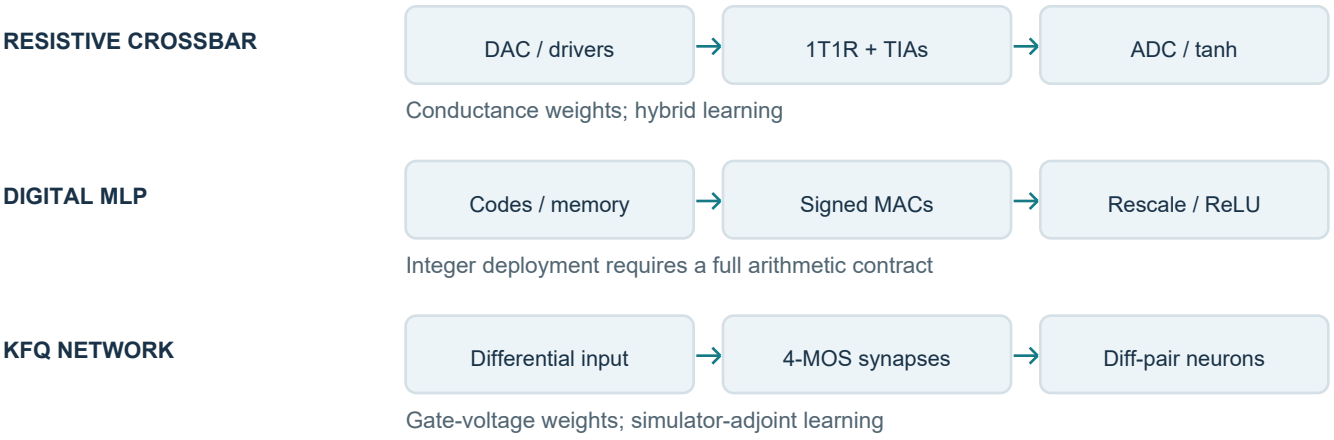
The research emphasizes original papers, author repositories, official benchmark definitions, device documentation, and the project's source code. The selected bibliography spans foundational results through recent large-system demonstrations. It is a targeted technical review, not a systematic meta-analysis of every published accelerator. Paywalled or inaccessible material is used only to the extent supported by an accessible original abstract or author version. No performance result from a publication is silently transferred to your technology or workload.

Four evidence labels are used throughout. Published hardware measurement means the cited authors measured fabricated devices or a chip. Published simulation means their numerical or architectural result. Local saved evidence means your existing logs, checkpoints, decks, or verification output. Proposed protocol means a recommendation that has not been executed. An adjoint simulator result is not an on-chip training demonstration.

The term diff_pair was resolved to the serial ring in toy2d_serial_diff and the differential/four-quadrant implementations in kfabric. No separately named diff_pair directory was identified in the inspected project. The report therefore covers both the ring and KFQ rather than guessing which one the term denotes. kfabric's recurrent-core document is discussed as design intent, because several scripts named there are absent from the current file inventory.

For comparisons, define the boundary before estimating a number. A core-only comparison excludes interfaces from all arms and cannot be advertised as application efficiency. An accelerator comparison includes conversion, buffering, bias generation, weight storage, reset, scheduling, and output handling. A host-inclusive comparison additionally includes training or inference software and communication. Different boundaries answer different questions.

2. What distinguishes the architectures



All three families obey Kirchhoff's laws. The differentiating question is what physical quantity represents a weight, where nonlinearity occurs, how intermediate signals are carried, and how learning modifies the system.

Architecture	Stored or controlled quantity	Forward operation	Present learning path
Resistive 1T1R crossbar	Two conductances per signed weight	Parallel current summation, transistor TIAs, software converters and tanh	Circuit forward, ideal-gain backward, abstract programming
Digital MLP	Quantized weight codes; float master weights during QAT	Software dequantized matrix products and ReLU; hardware arithmetic primitives separately	Software gradients and floating-point updates
KFQ, kfabric	Two gate-control voltages shared across four MOS devices per weight	Differential MOS current summation and differential-pair interlayer neurons	Transient adjoint in simulator and host Adam
Closed diff-pair ring	Per-stage load, tail, and reference biases	Initial-state-driven recurrent transient, then software classifier	Simulator circuit gradients plus learned digital readout

The resistive device retains a programmed state; a MOS gate voltage is a continuously supplied control signal unless a retention mechanism is added. A gate voltage source in SPICE is not a weight memory. SRAM, nonvolatile memory plus DACs, floating-gate storage, or sample-and-hold capacitors would have different area, precision, leakage, refresh, and training implications.

Do not confuse a differential signal with a differential weight. Paired conductances provide signed weights even with single-ended row inputs. KFQ cross-connects differential row signals and paired gate controls to reject the large common-mode product. These are related but different circuit implementations.

3. Analog crossbar operation: the physical computation

For an ideal read, current through cell i,j is the input voltage relative to the column reference multiplied by its conductance. The column collects all cell currents. A feedback amplifier holds that column approximately at the reference, preventing the summing node from becoming an uncontrolled voltage divider.

$$I_j = \sum_i G_{ij} (V_i - V_{ref})$$

$$V_{out,plus} - V_{out,minus} = -R_f \sum_i (G^{plus}_{ij} - G^{minus}_{ij})(V_i - V_{ref})$$

Positive resistive devices cannot directly store a negative conductance. Differential cells encode the signed quantity through a difference. One zero weight can have many common-mode conductance choices. They produce the same ideal differential signal but different total current, energy, selector error, and noise sensitivity. Therefore signed weight precision and the common-mode conductance policy should both be reported.

Your exported layer-1 decks have three rows, including bias, and sixteen column pairs. Layer 2 has seventeen rows and two pairs. Their row-source swing is 0.3 V around a 1.0 V reference. Every selected cell consists of an NMOS in series with a resistor whose value is the programmed model state. The selectors are access devices, not the stored synaptic weights. Each physical column has a seven-transistor, two-stage TIA, a 2 kOhm feedback resistor, and a 10 fF column capacitor. These facts come from the exported decks, not from a generic illustration.

Resistance is used as the SPICE representation of a conductance. That does not establish an RRAM compact model, switching mechanism, retention time, endurance, or a foundry-integrated memory process. Those require a chosen device technology and characterization.

4. Crossbar interfaces, addressing, and scaling

A hardware tile needs input storage, DAC or pulse-based row encoding, word-line drivers, read/program switching, column selection, readout, conversion, and output handling. For a parallel matrix-vector read, all participating rows can be enabled simultaneously. Row and column decoders are important for cell access, programming, verification, and shared-resource scheduling; one-hot row selection is not the entire MVM read protocol.

An ADC can be dedicated to each output or shared through a multiplexer. A physical column may have its own TIA even when converters are shared. Differential subtraction can occur before conversion or after two conversions; the cost, common-mode requirements, and error model differ. Your code differences two simulated TIA voltages before applying one software ADC model per logical output, so a physical implementation must match that choice or demonstrate equivalence.

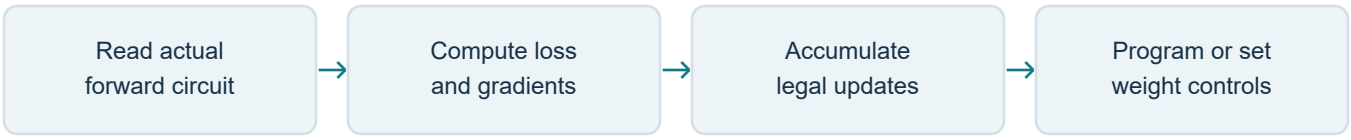
Scaling the array raises wire resistance, column loading, accumulated current, voltage-drop error, and ADC range demands. One useful experiment is to hold normalized weights fixed while increasing fan-in, then measure ideal-versus-circuit residual, signal-to-noise ratio, selector voltage drop, and converter clipping. Parallelism does not make these errors disappear.

NeuRRAM combines RRAM arrays, configurable sensing/conversion, peripheral control, and hardware-aware calibration, and reports hardware-measured task results. Its reported peak MVM efficiency uses a specified utilization and excludes some intermediate-transfer costs. It provides evidence that peripheral organization matters; it is not a measured proxy for your current-loaded TIA tile. [R5]

ISAAC and PUMA are architectural studies integrating analog arithmetic with digital support, tiling, and scheduling. Their modeled accelerator results should not be presented as measurements of a passive array or as evidence that the local Python programming model has equivalent overhead. [R14, R15]

Recent work reinforces the need to identify the boundary precisely. A 2024 residue-number architecture composes higher precision from multiple low-precision analog operations; it is a different architectural option, not an implemented improvement to your tile. A 2025 PCM-chip ALBERT study places the model's fully connected weights on hardware while several attention and activation computations remain in software. Such a hardware/software experiment should not be called a wholly end-to-end analog accelerator. These examples also make blanket claims that analog hardware only handles tiny easy tasks obsolete. [R29, R30]

5. Crossbar training: three distinct workflows



A simulator can expose gradients that a physical controller cannot obtain without additional circuitry.

Ex-situ training first learns a software network, maps its weights into legal conductance ranges, programs the array, and tests the resulting circuit. Noise-aware software training and chip-in-the-loop fine-tuning can compensate for important device and circuit errors. They are practical baselines for your project because they separate the quality of the inference hardware from the difficulty of its update rule.

Hybrid in-situ learning repeatedly performs forward calculations with the array, computes error and gradients in a digital controller, and applies physical programming updates. Training can adapt to the actual device state even when the gradient is approximate. The cost includes controller work, state readback, addressed writes, and any verification. Calling this in-situ does not imply that all backpropagation arithmetic occurs in the analog array.

Fully analog training architectures additionally use transposed MVMs and pulse coincidences for local outer-product updates. This requires backward-read access, appropriate device-update behavior, and a hardware protocol for matching desired updates to conductance changes. A sign-only host update is a particular algorithmic choice, not an unavoidable property of resistive hardware.

Li et al. demonstrate 1T1R in-situ SGD using a two-pulse conductance-setting scheme and software ReLU; their experiment reports 91.71% on a separate 10,000-image MNIST test set. The larger accuracy reported for a larger array is simulation. This directly contradicts the local universal claims that proportional updates are unbuildable, all accurate crossbar training is Manhattan, and ReLU is unsuitable for differential arrays. [R3]

The passive MLP of Merrikh Bayat et al. uses differential synapses, bias inputs, tanh-like analog neurons, and computer-assisted updates. Its fixed-amplitude in-situ experiment is relevant to your Manhattan ablation, but its passive array and analog interlayer activation differ from your 1T1R/software-activation implementation. [R2]

6. Updates, granularity, and programming error

For a desired weight increment, a controller can raise the positive conductance, lower the negative one, or combine both operations. Raising one while lowering the other produces twice the normalized weight increment when both move by the same normalized amount. This factor must be reflected in the learning-rate definition. Rail handling also matters: an update removed from a residue accumulator may not have been physically realized after clipping.

The current Manhattan code multiplies its modeled conductance step by a fractional pulse count, then quantizes and adds programming noise. This is not an integer pulse scheduler. Focused checks earlier in this chat showed a sub-level update disappearing without noise and a zero-pulse request still changing conductance when noise was enabled. An implementable schedule would use integer pulses, accumulated update residue, probabilistic pulse issuance, or characterized variable pulse amplitude/duration.

The current writeverify path accumulates proportional updates until a level threshold is crossed. However, any nonzero update causes write() to run over the entire layer arrays; zero-update cells are requantized and receive new noise. A masked write is necessary. Its verify flag merely changes Gaussian error standard deviation; it does not execute read-compare-correct cycles and does not guarantee a maximum error. This path bypasses the nonlinear pulse law, so its accuracy cannot validate that law.

The local 100:1 conductance window, 64 target levels, and programming-error values are assumed model parameters rather than a characterized universal device. Noise is added after target quantization, so the simulated final conductances are not confined to exactly 64 values. Logged conductances are assigned model states, not independently measured physical cells. Higher state counts have been demonstrated with specific programming methods, but that does not establish equivalent fast incremental-update precision. [R28]

The crossbar backward path uses ideal layer gain and programmed weights, rather than the transistor network's derivative. It effectively ignores quantizer discontinuities and does not explicitly mask ADC clipping. That can be a useful surrogate, but gradient sign is not guaranteed near clipping or when circuit errors alter the transfer. Compare it with finite differences of a smoothed or appropriately defined circuit objective, and separately track saturated samples.

Nandakumar et al. provide a strong precedent for high-precision digital update accumulation with imprecise analog changes. Their demonstrated update transfer uses blind pulses rather than iterative write-verify. Keep accumulator precision, write-verify policy, and device nonlinearity as separate ablations. [R7]

Gokmen and Vlasov identify device requirements for resistive training; Tiki-Taka addresses asymmetry through an alternative update architecture. Neither supports treating one poor sign-update schedule as a fundamental ceiling for crossbar learning. [R6, R8]

7. Digital MLP operation and an implementable integer path

A conventional MLP computes an affine transformation, a nonlinear activation, and another affine transformation. Bias is required in all arms for the toy circle task. Hardware can time-share one MAC, instantiate parallel lanes, or use an array. The algorithm does not prescribe the cheapest scheduling architecture.

```
z1 = W1^T [1, x]; h = activation(z1); scores = W2^T [1, h]

real_value = scale * (integer_code - zero_point)

acc_j = bias_int_j + sum_i qx_i * qw_ij
```

An integer deployment contract specifies operand signedness, input and output scales, accumulator width, bias scale, rounding, saturation, and the final shift or multiplier. A conservative accumulator bound is based on the sum of worst-case product magnitudes plus bias. Using INT8 weights does not mean that sums or biases should also be INT8. For symmetric codes with magnitude at most 127 and fan-in N, sum magnitude can approach N*127*127.

Jacob et al. explicitly co-design simulated-quantization training with integer-only inference, including wider accumulation and bias handling. This correspondence is the right standard for your digital arm. It does not require reproducing their MobileNet or affine zero-point scheme; a symmetric power-of-two format is acceptable if its deployed computation is fully specified and checked. [R11]

Input and output interfaces should depend on the application boundary. With digitally supplied datasets, analog arms need input encoding and output digitization or comparators. With analog sensors, the digital arm may require an input ADC. Use the same task boundary for every arm rather than charging conversion only to the competitor.

8. Digital QAT and the current implementation gaps

Quantization-aware training typically retains floating-point master parameters while simulating quantization in forward computation. A straight-through estimator supplies a useful approximate derivative through rounding. It is not the mathematical derivative of a discontinuous quantizer. Clipping derivatives and scale adaptation should be declared because they influence the optimization problem. [R12]

Your digital_train.py uses rounded, dequantized weights and a quantized hidden activation. It leaves the first input in floating point and performs matrix products in floating point. Its hidden activation scale is recalculated separately for each sample. Export writes integer weights and weight scales, but not a complete fixed input/activation/bias/accumulator specification. Therefore the reported accuracy is fake-quantized software accuracy, not established bit-exact accelerator accuracy.

An effective repair is to choose scales using training/calibration data, freeze them for testing, implement a separate integer forward path, and compare per-layer integer traces against the fake-quantized path. Dynamic per-sample scaling is also implementable, but then the scale determination, its precision, and its runtime hardware cost must be included. Backpropagation currently uses float master weights in the hidden error calculation; record this as an approximate QAT gradient rather than claiming exact differentiation of the deployed integer model.

Do not use an intentionally inefficient ripple multiplier as the sole digital competitor. EIE is directly relevant to fully connected inference and weight/data movement; Eyeriss demonstrates dataflow-aware energy accounting; TPU demonstrates the role of memory and utilization in a dense MAC architecture; CMSIS-NN supplies a practical embedded software baseline. These are implementation alternatives, not portable energy constants. [R13, R16-R18]

9. Digital circuit verification: what is actually available

digital_netlist.py constructs static CMOS gates, four-NAND XORs, a full adder, ripple adders, and an unsigned array multiplier. The full adder uses eleven NANDs and forty-four MOSFETs. The two-bit ripple adder uses eighty-eight MOSFETs; the 2x2 multiplier uses one hundred twelve. A complete trained network, SRAM macro, sequential accumulator/registers, clock network, signed multiplier wrapper, and control path are not present in these files.

The existing verification log records all eight full-adder cases and all sixteen two-bit adder cases as correct. The multiplier converged for fourteen of sixteen inputs and was correct for those fourteen. Passing all converged cases is useful evidence; it is not exhaustive electrical verification of every input. Unresolved cases must stay visible in a coverage denominator. Boolean checks performed for the earlier schematic export passed the small unsigned primitive truth tables, but they do not resolve transistor-level nonconvergence.

The array multiplier interprets its operands as unsigned. Negative exported weights therefore require signed arithmetic handling. Directly multiplying the two-bit code for -1 by +1 with the existing primitive yields the unsigned value 3. Functional verification should include the signed extreme values, bias addition, rounding ties, overflow, and saturation.

The source structure gives an N-bit unsigned multiplier transistor count of 6*N*N + 44*N*(N-1), or 2,848 devices at N=8, before signed handling or registers. The approximate 3,800 figure in the documentation is not the exact count of this generator. A measured solve ceiling from a few input patterns is not a universal simulator capacity limit.

10. KFQ: four-quadrant MOS multiplication

The implemented KFQ is a feedforward network with four cross-connected NMOS devices per logical weight. Inputs use paired voltages about a common mode. Two gate controls are swapped across the two column halves. Resistors pull the columns toward VDD, and differential-pair neurons drive the next layer. The final classifier reads the last column differences rather than the last differential-neuron outputs.

```
vplus = vcm + swing*x/2; vminus = vcm - swing*x/2

delta_I = -beta*(theta_a - theta_b)*(vplus - vminus)

delta_Vcolumn = Rload*beta*(theta_a - theta_b)*(vplus - vminus)
```

The second equation follows by expanding four matched square-law currents. The squared-input terms and common-mode offset cancel when all devices remain in the assumed saturation regime and have the same ideal parameters. The resistive-load voltage difference reverses the current-difference sign. This is an original derivation from the local topology, not proof of exact multiplication for arbitrary MOS models.

The single-ended two-transistor KDN leaves a weight-dependent offset tied to the large input common mode. KFQ differential row encoding removes that term in the ideal algebra. Bias must then be deliberately reintroduced by a fixed differential reference row. With zero biases and odd activations, a network has strong symmetry restrictions; the practical test is a matched bias/no-bias ablation, not a universal inference from one failed circle run.

At matched 2-8-2 sizes, KFQ has forty-two logical synapses: 168 synapse NMOS devices plus thirty neuron/tail devices, totaling 198 MOSFETs. Its two gate controls per weight plus two tail controls give eighty-six source parameters, although the tail controls are frozen by default during training. Device count is not layout area or storage cost.

The writer also instantiates final-layer differential-neuron circuits even though class scores are read from their input columns. Include those devices in the as-built inventory. Removing an unused final-stage circuit is a possible optimized variant, but its loading and predictions must be rechecked before assigning its lower cost to the original design.

11. Differential-pair neurons and operating constraints

A transistor differential pair is a smooth bounded current-steering element when biased appropriately. In weak inversion its ideal transfer is tanh-like. In a matched strong-inversion square-law model, its two-conducting-device transfer instead has a different form:

```
delta_I = beta*v_diff*sqrt(I_tail/beta - v_diff^2/4)
```

This relation holds only inside the two-device conduction range; outside it, one device carries essentially the tail current. Real output voltage depends on loads, finite output resistance, tail compliance, and the next stage. The local Level-1 model is not a weak-inversion compact model, so labeling its neuron as an exact tanh is inappropriate. Measure its actual transfer and derivative at the operating point. [R25]

Conduction requires positive gate overdrive at every relevant input and state. Saturation requires adequate drain headroom. Tail current affects both nonlinear gain and the output common mode; changing it can move the next layer out of its conduction window. kfq_spec.py and the train_kfq.py comments already identify this coupling. A single bias or load value is generally inadequate as fan-in and interlayer common mode change.

A useful diagnostic sweep varies input difference, input common mode, paired gate difference, paired gate common mode, and load. Record output difference, common mode, device operating regions, incremental gain, and static current. Repeat after mismatch and realistic device effects are enabled. Observe whether useful signal survives depth rather than only checking that the output is nonzero.

12. Adjoint training and hardware realizability

KFQ training runs a transient forward/adjoint solve, reads class-score sensitivities with respect to gate-control sources, and applies host-side Adam. For cross-entropy with a fixed voltage-to-logit gain, the parameter gradient combines the loss derivative and each differential output sensitivity.

```
dL/dtheta = gain * sum_j (p_j - target_j) * (dVcp_j/dtheta - dVcn_j/dtheta)
```

The gain does not change argmax, but it does change loss curvature and gradient scale. Calibrate it on training data and freeze it for a comparison unless adaptive calibration is explicitly part of the design. Frozen tail sources should not count as actively trained parameters simply because sensitivities are requested for them.

A simulator adjoint can provide derivatives of a mathematical circuit model efficiently. It does not automatically specify an electrical on-chip adjoint network, how internal states are retained, how a loss is injected, or how weights are programmed. Code that returns sensitivities for many outputs in one API call may perform multiple internal solves; measure solver factorization, right-hand-side work, storage, and total runtime rather than equating calls with constant cost.

For model validation, use central finite differences over several step sizes, random parameters, and samples. Check scale-sensitive relative and absolute error and directional derivatives, not only vector cosine. Ensure missing sensitivities fail loudly; current wrappers silently substitute zero for absent parameters. Quantized programming and clipping require separate treatment from a differentiable continuous-parameter model.

Finite differences establish agreement with the discretized simulator map. They do not establish that the map accurately resolves the physical transient. Repeat after timestep refinement and under both supported integration methods; verify the initial-condition and charge-history semantics as well. The inspected transient engine first solves a DC bias point, then overrides selected nodes with .IC values before initializing transient states. That behavior must be reproduced when porting the ring to another engine.

KirchhoffNet and Neural ODE work support the dynamical-model and adjoint viewpoint. Physics-aware training supports hardware-forward/surrogate-backward learning as another practical route. Equilibrium propagation is an alternative only when its dynamical assumptions are satisfied; it cannot simply be substituted into an arbitrary nonreciprocal transistor network. [R1, R9, R10, R19]

13. The ring, recurrent-core proposal, and published KirchhoffNet

The serial ring explicitly closes a chain of ten differential-amplifier stages. Data is encoded into initial conditions on storage-capacitor nodes, the circuit evolves for 100 ns, and a learned software affine readout predicts the class. It has fifty MOSFETs and ten 1 pF capacitors, thirty circuit-bias parameters, and twenty-two trainable readout values for binary classification. This is not the same experiment as direct column argmax in KFQ.

The ring trainer standardizes features from initial training states, trains the readout freely, and accepts guarded circuit updates. It skips nonconvergent samples and scores only the surviving subset. For a fair report, preserve the original test denominator and treat failed predictions as failures, or report valid-only accuracy alongside coverage and effective accuracy. A good learned readout can mask a weak or nearly frozen circuit; compare frozen-circuit and trained-circuit cases with the same readout training budget.

The architecture document proposes a small recurrent core reused across T input chunks, with state supplied through differentiable voltage sources and BPTT through state-transfer Jacobians. It names scripts that are absent from the current inventory. Its architectural ambitions and parameter counts should not be attached to existing feedforward KFQ results. If implemented later, reset, sample-and-hold, state transfer, and the T-step latency become part of the measured system.

An initial-state sensitivity exists mathematically for an ODE. The document's statement that an .IC directive makes BPTT impossible is therefore an API limitation, not a fundamental theorem: source-driven states expose derivatives through the available simulator interface, while a different solver could provide initial-condition Jacobians directly. Likewise, Perron-Frobenius intuition about repeated positive linear operators does not by itself prove inevitable rank collapse in every nonlinear, layer-varying positive-weight network. Keep those explanations as hypotheses tested by the local diagnostics.

The published KirchhoffNet formulates nonlinear circuit-inspired ODE dynamics and considers topology changes across depth. Its capacitance-based time rescaling is a theoretical argument with explicitly acknowledged hardware limits, not a fabricated femtosecond inference measurement. Your CMOS ring and KFQ are related physical modeling explorations, not identical transistor realizations of every published edge law. [R1]

14. The simulator fidelity finding

The inspected simulator routes LEVEL=1 MOS instances to models/mosfet_level1_model.py. Its compute() uses VTO, KP, W, and L, implements a piecewise square-law current, assigns saturation output conductance near 1e-12 S, sets gmb to zero, and returns zero terminal charges. It does not use the supplied GAMMA, PHI, or LAMBDA values. This is a source-code observation; it is not inferred from a suspicious plot.

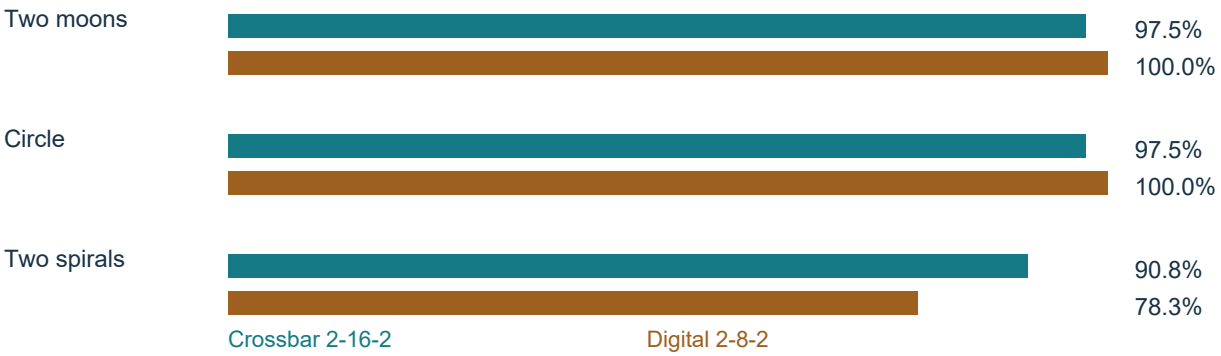
Consequences include absent body effect, absent modeled channel-length modulation, absent intrinsic gate/drain charge, no modeled subthreshold current, and highly ideal saturated current sources. The algebraic four-quadrant cancellation is consequently tested in a particularly favorable model. TIA gain, interlayer compliance, switching energy, and transient delay cannot be promoted to realistic silicon results on that basis alone.

In a more realistic model, source-body-dependent thresholds change the input law; mismatch produces offsets and residual terms; drain-voltage dependence couples the nominal multiplier to column loading; gate and junction capacitances add poles and energy. First-order body-effect gain around a common mode can be derived from the threshold-versus-source-voltage function, but only a chosen compact model and operating point determine the actual error. Do not silently apply a generic body-effect correction to results from a simulator that omitted it.

Action: validate the active model with device-level DC and charge tests, then port a representative slice to an independent SPICE engine and a characterized model set. A different nominal technology is a sensitivity experiment, not proof that the original design has that foundry's power or area. Model cards and geometry alone do not establish a foundry node. [R23, R24]

15. Existing results: useful evidence, incompatible experiments

Local final accuracies: different widths / protocols, not a ranking



The following table recalculates final test accuracy from local logs. It is a historical inventory, not an architecture ranking. All refdesign runs use one recorded initialization seed and 120 test points; the larger KFQ circle run uses 300 test points. Epoch-by-epoch test maxima are selection-biased when used for tuning or checkpoint choice.

Task	Older crossbar, 2-8-2 Manhattan	Newer crossbar, 2-16-2 update accumulator	Digital, 2-8-2
Two moons	85.0% final	97.5% final	100% final
Circle	44.2% final	97.5% final	100% final
Two spirals	66.7% final	90.8% final	78.3% final

The crossbar comparison changes width, update rule, programming-error assumption, activation slope, and training duration. It cannot isolate a device effect or prove a crossbar advantage on spirals. The README's circle-failure narrative refers to the old run, not the final trained exports.

The saved KFQ circle checkpoint uses 2-8-2, 300 train/300 test, with a final re-evaluation of 99.0%. Its small 4x4 MNIST experiment reports 67.0% test accuracy versus 76.0% for raw-feature logistic regression on that benchmark. This identifies a useful scaling weakness; it is not comparable with published full-size MNIST results.

At 120 test points, a 97.5% score has a Wilson 95% interval approximately 92.9-99.1%; 100% has a lower bound near 96.9%. Repeated evaluations of the same examples do not enlarge the independent sample count. The current convergence helper returns the first epoch reaching 95% of a run's maximum, while the trainer labels it as the best epoch. Store the true maximum epoch separately.

16. A fair comparison requires three experiment tracks

Track A isolates inference implementation. Match topology, biases, activation, input normalization, output decision, and a common software-trained function as closely as each hardware permits. Compare float software, integer software, ideal crossbar, transistor crossbar, and measured-transfer KFQ. Where KFQ cannot realize the exact activation, report a transfer mismatch and compare to its own software control as well. This track attributes accuracy loss to implementation.

Track B measures learning capability. Each arm receives its appropriate training mechanism under the same dataset, validation policy, and explicit tuning budget. Separate crossbar ex-situ mapping, crossbar sign updates, accumulated pulse updates, and idealized target writes. Separate KFQ host-adjoint learning from a proposed on-chip method. Report achievable accuracy and total training work, not merely equal epoch counts.

Track C measures deployable hardware at target accuracy. Allow each arm to choose width, precision, and scheduling within a declared design-search budget. Compare energy, latency, area, and robustness at a common minimum task quality. Plot Pareto fronts instead of declaring the smallest core the winner. A streaming recurrent design belongs in this track only after its complete T-step schedule and state interface are modeled.

The matched-architecture and best-deployable-system questions are both valuable. They should be separate figures. For example, 2-8-2 and 2-16-2 can be common experimental configurations, but a wide crossbar should not be compared to a narrow digital model solely because those happen to be the saved checkpoints. All bias coefficients, paired controls, fixed controls, and learned readout parameters need explicit counting.

17. Data, tuning, and statistical testing

Create one immutable dataset artifact containing raw examples, labels, train/validation/test indices, and generator seed. Fit every data-dependent transform using training data only. Hash the artifact and include its digest in every run. The current toy loaders standardize the full generated pool before splitting. For the 2D linear scaling followed by training-only min-max mapping, some effects may algebraically cancel, so the observed scores should not automatically be called inflated; the correct general protocol is still split first and fit transforms only on training data. [R26]

Use validation data for architecture selection, learning-rate schedules, ADC range/slope tuning, operating-point search, and checkpoint selection. Test once after selection. A fixed test set repeatedly used to choose settings is no longer an untouched test set. Initialization seeds alone do not vary the dataset: the current toy loader's seed is fixed separately from the training seed. Vary and record both.

Recommended staged budget: three data splits by five initialization seeds for the initial toy study, then additional repeats if the paired difference is small. This is a proposed resource allocation, not a statistical sufficiency guarantee. Report the distribution of seed-level results, uncertainty across splits, and paired prediction disagreements. For two fixed models tested on the same samples, a paired test such as McNemar's test uses the disagreements; independent confidence-interval overlap is not the best comparison. Cluster resampling by split or seed when estimating uncertainty over repeated experiments. [R20]

For simulator failures, report coverage and effective accuracy using all submitted samples. Define a nonconvergence retry policy in advance and preserve every failure, rather than letting a favorable subset decide quality. For reliability testing, separate chip-to-chip mismatch, run-to-run noise, and programming noise; each is a different source of variation.

18. Verification ladder before benchmarking

Gate	Minimum check	Evidence to preserve
Device physics	MOS region boundaries, output conductance, body effect, charge and capacitance; resistive read/write law	Models, DC curves, charge checks, simulator versions
Primitive function	NAND/adder signed arithmetic; differential cell multiplication and sign	Complete truth-table coverage, residuals, missing cases
Analog layer	Input/gate/common-mode sweeps, fan-in sweep, selector error, TIA stability and saturation	Voltages, currents, operating regions, conversion clipping
Gradient	Finite-difference sweep and random directional derivatives	Absolute/relative error, cosine, unavailable sensitivity flags
Network inference	Exported checkpoint reproduces predictions; integer traces match contract	Per-sample predictions, scale metadata, split hash
System cost	Transition-driven timing and energy; realistic memory/interface model	All source currents, reset/hold/program overhead, load sweep

For crossbars, verify the actual seven-transistor readout, not only an ideal controlled source. Test offset at zero differential weight, common-mode conductance, and large fan-in. Your existing selector-removal and ideal-MVM tests are useful small checks, but they do not establish wire-parasitic or nonlinear-device behavior at scale.

For KFQ, measure rank, singular values of the input-output Jacobian, signal variation across samples, common-mode drift, and the fraction of conducting devices as depth increases. A high affine fit of the MVM stage is desirable; a high affine fit of the entire classifier may indicate inadequate nonlinearity. Training-parameter displacement alone does not prove useful learning; compare against a frozen-circuit readout control.

For digital logic, verify the complete signed layer operation, including bias, rescale, ReLU, and output accumulation. Cross-check the same component with an independent engine if nonconvergence prevents coverage. If the complete datapath remains unavailable, report software quality and modeled hardware cost as separate evidence categories.

19. Energy, power, and training cost

```
E_inference = integral_0^T sum_k [-V_k(t)*I_k(t)] dt + E_interfaces + E_storage + E_control

E_average = E_inference + E_program/N_reuse + P_standby*T_idle_per_sample
```

The sign convention must be verified against a known resistor and capacitor test. Distinguish supply energy delivered, energy returned to sources, and positive consumption. Negative bias-source contributions are not evidence that a deployable driver recovers energy unless that driver is modeled. Include the initial-state injection and reset path for the ring; presetting capacitors with solver directives is not a free hardware operation.

Measure actual sample-to-sample transitions after realistic input setup, not only startup from an arbitrary solver seed. Analog bias current continues until power gating is applied; shortening an integration window does not implement a power gate. Include hold and readout time. For digital hardware, integrate clocked operand activity with representative toggles and account for SRAM accesses, clocking, registers, and leakage. A DC truth table cannot yield switching energy.

The converter constants in compare/tech.py are assumptions: the 100 fJ/step ADC figure gives 25.6 pJ for one nominal 8-bit conversion. Eighteen differential outputs in the 2-16-2 crossbar would then imply 460.8 pJ if each pair uses one such conversion; thirty-six single-ended conversions imply 921.6 pJ before subtraction. These are arithmetic sensitivity examples, not measured circuit energy or proof that every ADC has that cost. Use an implementation with stated ENOB, sampling rate, range, and technology. [R22]

Report inference and training separately. Crossbar training includes reads, writes, verifies, refresh, and controller gradients. KFQ simulation-adjoint training includes host work; physical online training would additionally need a derivative/programming mechanism. Device programming energy amortizes with reuse; a recurrent design multiplies per-step costs by T.

20. Latency, throughput, and area

Latency is elapsed time from a defined valid input to a valid output. Analog array settling is only part of it. Add input setup, converter latency, digital activation, interlayer sequencing, reset, and final decision. A shared ADC trades area against sequential conversion time. Pipeline throughput can exceed reciprocal single-sample latency; report both with utilization and batch size.

The local default 0.1 fF KFQ node load is an explicit numerical assumption. Evaluate 1, 10, and 100 fF external-load scenarios, then include extracted intrinsic and interconnect capacitance for a physical claim. Those exploratory loads are not substitutes for extraction. A 100 ns simulation stop is not a measured inference time, and a tiny RC value is not an end-to-end speed measurement.

The current settling helper returns the last out-of-band sample. Strictly, the guaranteed in-band time lies after that sample; return the next in-band timestamp or an interpolated crossing with a convergence check. Its relative-to-final-swing band may look acceptable while a classification margin is tiny. Define numerical validity in output-code or class-margin units as well, and repeat with a finer time grid.

Area must include weighted device geometry, resistors, capacitors, storage, routing, and interfaces. The formula n_devices() in KFQ counts MOSFETs only, whereas the crossbar helper mixes selectors, cell resistors, and readout devices; their totals are not comparable area metrics. In the matched 2-8-2 core, the explicit crossbar has 224 MOSFETs plus 104 resistors and 20 capacitors, compared with KFQ's 198 MOSFETs plus 40 resistors and 40 capacitors. Different device widths and missing support circuits can reverse a count-based intuition.

Do not price SPICE resistive cells as large poly resistors and simultaneously describe them as compact RRAM without declaring separate technology cases. Nominally passive fixed poly is not a programmable synapse. A realistic RRAM area model needs its own integrated-memory process and access devices.

21. Robustness and the experiment matrix

Sweep	Crossbar	Digital MLP	KFQ / ring
Precision	Conductance levels, residual programming spread, DAC/ADC bits	Weight, input, activation and accumulator bits	Gate-control quantization and drift, input/output resolution
Physical variation	Selector/TIA mismatch, device variation, stuck cells, wire resistance	PVT, timing slack, SRAM behavior, voltage scaling	MOS mismatch, load/tail mismatch, source noise, bias drift
Dynamics	Read disturb, relaxation, programming endurance	Clock rate, switching patterns, power gating	Load capacitance, reset state, settling margin, recurrent stability
Algorithm	Mapping, noise-aware training, pulse/update method	QAT versus post-training quantization, scale policy	Frozen versus trained circuit, tail policy, readout policy

Use device-appropriate variations, not identical percentage noise in incompatible physical quantities. Calibrate distributions from a selected technology where possible. Threshold mismatch scales with geometry in established matching models, but a coefficient from another process is not your coefficient. Separate exploratory mismatch scenarios from characterized Monte Carlo models. [R21]

For each candidate, retain per-sample predictions and class margins. Sweep accuracy against energy, area, latency, and precision. Report worst corner as well as median; retain tails of the failure distribution. Demonstrate whether hardware-aware retraining recovers losses rather than comparing a robustly trained candidate with an unadapted competitor. Published hardware-aware training studies show that the location and form of nonidealities matter; use them to structure an error budget rather than as a guarantee of robustness. [R4]

The current architecture has no established measured full-system advantage. That is an experimental opportunity: test whether fewer internal conversion boundaries remain beneficial after gate-bias storage, common-mode control, buffering, and realistic loads are charged to KFQ.

22. A concrete implementation and publication roadmap

Stage 1 - restore numerical credibility. Fix the crossbar write mask, zero-pulse handling, update scaling and rail residue accounting. Rename the residual-noise model accurately. Implement the digital signed integer deployment contract. Make unavailable sensitivities and incomplete logic coverage explicit failures. Replace test-selected checkpoints with validation-selected ones.

Stage 2 - create one benchmark harness. Save shared raw splits and transforms, a common 2-8-2 and 2-16-2 topology set, per-arm exported checkpoints, and per-sample predictions. Retain float controls with the actual activations of each circuit. Freeze ADC ranges and operating points for testing. Use the older ring only with frozen-circuit/readout ablations and a fixed failure denominator.

Stage 3 - validate real circuit assumptions. Test the active MOS implementation, port a small cell/layer to independent SPICE, and add realistic output resistance, body effect, charge, loads, and mismatch. Size the TIA and KFQ interlayer operating points under these effects. Decide how every trained gate control is stored and reproduced on hardware.

Stage 4 - measure deployed cost. Define the digital workload boundary, sequential/parallel schedules, converters, sample-and-hold, power gating, and startup conditions. Compute per-inference energy on a representative input trace; use SRAM, DAC, and ADC costs with documented assumptions and sweeps. Preserve core-only and system totals separately.

Stage 5 - extend the tasks. Start with the three toy sets and Iris, then compare identical 4x4 and 8x8 digit inputs on much larger independent test sets. A reduced-MNIST experiment is a scaling probe, not a benchmark comparable to the standard full-resolution test set. For stronger deployment evidence, use a documented accuracy/latency/energy workload inspired by MLPerf Tiny, without claiming an official submission unless its rules are followed. [R27]

Publish a Pareto comparison at matched quality, including unsuccessful designs and realistic uncertainty. A useful result can be a verified tradeoff rather than a universal winner: reduced internal conversion overhead versus bias-storage, headroom, precision, and static-current costs.

23. Suggested reporting template and decision criteria

Every table row should state architecture/version, evidence level, dataset fingerprint, input resolution, train/validation/test counts, seeds, topology, logical weights, physical devices, trainable controls, precision, activation, training method, and checkpoint-selection rule. Its quality fields should include test accuracy, confidence interval, failure coverage, and class-margin distribution.

Its hardware fields should include technology/model identity, VDD, bias settings, modeled memory/interfaces, load/parasitic assumptions, core energy, interface energy, storage/control energy, programming energy, standby policy, end-to-end latency, throughput, and area methodology. Never leave a missing block as a silently zero contribution. Use not modeled, assumed, simulated, or measured to distinguish the entries.

Record training cost using the processed examples, forward/backward calls, matrix factorizations, gradient right-hand sides, host runtime, write pulses, verification reads, and calibration passes. Report a validation-defined target crossing rather than each run's own test maximum as the convergence target. If one arm never reaches the target, retain that outcome.

A defensible architectural win requires comparable task quality; an explicit common boundary; robust operation under the stated model/corners; and a cost benefit that survives plausible peripheral assumptions. If the core-only advantage disappears when gate-voltage storage is included, report that finding. If KFQ is better only at low input precision or shallow depth, identify that operating regime. The objective is to locate the useful design region, not to make every comparison favor the proposed circuit.

24. Annotated reading guide

R1 establishes the circuit-inspired ODE formulation and its timing assumptions; compare its edge functions with your transistor implementation. R2 and R3 are complementary early learning demonstrations: passive tanh-like MLP versus 1T1R with software ReLU and a two-pulse update scheme. R4 and R5 connect device/circuit errors to inference quality and complete chip organization. R6-R8 focus on training-device requirements and algorithms for limited, stochastic, asymmetric updates.

R9-R10 explain differentiable dynamical and physical-learning viewpoints; R19 explains when an energy-based alternative to backpropagation is applicable. R11-R12 explain deployment-compatible quantization and approximate quantizer gradients. R13-R18 prevent the digital comparison from being reduced to a slow multiplier: memory organization, dataflow, utilization, software kernels, and sparse fully connected inference matter.

R20-R21 support repeatable benchmarking and variation modeling; R22-R26 support converter assumptions, device models, physical loading, and leakage-free preprocessing. R27 supplies a benchmark framework around accuracy, latency, and energy. R28 illustrates why demonstrated conductance precision must be tied to a specific programming method and should not be conflated with fast incremental training precision.

The practical reading order is R1, R3, R5, R7, R11, R4, R20, and R27, followed by the architecture and circuit-validation references relevant to the implementation you choose. Each source's claims should stay at its own evidence level.

R29 and R30 extend the reading to 2024-2025: precision architecture and a substantially larger hybrid hardware experiment. Their relevance is methodological, not a recommendation to replace the present small classification study with a Transformer benchmark.

References

[R1] Gao, Z., Sun, F.-K., Rohrer, R., and Boning, D. S. KirchhoffNet: A Scalable Ultra Fast Analog Neural Network. ICCAD, 2024. DOI 10.1145/3676536.3676662. https://arxiv.org/html/2310.15872 ; author code: https://github.com/zhengqigao/kirchhoffnet

[R2] Merrikh Bayat, F., et al. Implementation of multilayer perceptron network with highly uniform passive memristive crossbar circuits. Nature Communications 9, 2331, 2018. https://www.nature.com/articles/s41467-018-04482-4

[R3] Li, C., et al. Efficient and self-adaptive in-situ learning in multilayer memristor neural networks. Nature Communications 9, 2385, 2018. https://www.nature.com/articles/s41467-018-04484-2

[R4] Rasch, M. J., et al. Hardware-aware training for large-scale and diverse deep learning inference workloads using in-memory computing-based accelerators. Nature Communications 14, 5282, 2023. https://www.nature.com/articles/s41467-023-40770-4

[R5] Wan, W., et al. A compute-in-memory chip based on resistive random-access memory. Nature 608, 504-512, 2022. https://www.nature.com/articles/s41586-022-04992-8

[R6] Gokmen, T., and Vlasov, Y. Acceleration of Deep Neural Network Training with Resistive Cross-Point Devices: Design Considerations. Frontiers in Neuroscience 10, 333, 2016. https://arxiv.org/abs/1603.07341

[R7] Nandakumar, S. R., et al. Mixed-Precision Deep Learning Based on Computational Memory. Frontiers in Neuroscience 14, 406, 2020. https://www.frontiersin.org/journals/neuroscience/articles/10.3389/fnins.2020.00406/full

[R8] Gokmen, T., and Haensch, W. Algorithm for Training Neural Networks on Resistive Device Arrays. Frontiers in Neuroscience 14, 103, 2020. https://www.frontiersin.org/journals/neuroscience/articles/10.3389/fnins.2020.00103/full

[R9] Chen, R. T. Q., Rubanova, Y., Bettencourt, J., and Duvenaud, D. Neural Ordinary Differential Equations. NeurIPS, 2018. https://arxiv.org/abs/1806.07366

[R10] Wright, L. G., et al. Deep physical neural networks trained with backpropagation. Nature 601, 549-555, 2022. https://www.nature.com/articles/s41586-021-04223-6

[R11] Jacob, B., et al. Quantization and Training of Neural Networks for Efficient Integer-Arithmetic-Only Inference. CVPR, 2018. https://arxiv.org/abs/1712.05877

[R12] Bengio, Y., Leonard, N., and Courville, A. Estimating or Propagating Gradients Through Stochastic Neurons for Conditional Computation. arXiv, 2013. https://arxiv.org/abs/1308.3432

[R13] Han, S., et al. EIE: Efficient Inference Engine on Compressed Deep Neural Network. ISCA, 2016. https://arxiv.org/abs/1602.01528

[R14] Shafiee, A., et al. ISAAC: A Convolutional Neural Network Accelerator with In-Situ Analog Arithmetic in Crossbars. ISCA, 2016. DOI 10.1109/ISCA.2016.12. Corrected author version: https://nlp.cs.utah.edu/assets/pdfs/shafiee2016isaac.pdf

[R15] Ankit, A., et al. PUMA: A Programmable Ultra-efficient Memristor-based Accelerator for Machine Learning Inference. ASPLOS, 2019. https://arxiv.org/abs/1901.10351

[R16] Chen, Y.-H., et al. Eyeriss: A Spatial Architecture for Energy-Efficient Dataflow for Convolutional Neural Networks. ISCA, 2016. Author project and publications: https://eyeriss.mit.edu/

[R17] Jouppi, N. P., et al. In-Datacenter Performance Analysis of a Tensor Processing Unit. ISCA, 2017. https://arxiv.org/abs/1704.04760

[R18] Lai, L., Suda, N., and Chandra, V. CMSIS-NN: Efficient Neural Network Kernels for Arm Cortex-M CPUs. arXiv, 2018. https://arxiv.org/abs/1801.06601

[R19] Scellier, B., and Bengio, Y. Equilibrium Propagation: Bridging the Gap Between Energy-Based Models and Backpropagation. Frontiers in Computational Neuroscience, 2017; author preprint 2016. https://arxiv.org/abs/1602.05179

[R20] Bouthillier, X., et al. Accounting for Variance in Machine Learning Benchmarks. MLSys, 2021. https://arxiv.org/abs/2103.03098

[R21] Pelgrom, M. J. M., Duinmaijer, A. C. J., and Welbers, A. P. G. Matching properties of MOS transistors. IEEE Journal of Solid-State Circuits 24(5), 1433-1439, 1989. DOI 10.1109/JSSC.1989.572629. https://ewh.ieee.org/r5/denver/sscs/References/1989_10_Pelgrom.pdf

[R22] Murmann, B. ADC Performance Survey, ISSCC and VLSI Symposium. Maintained author dataset, accessed 12 October 2026. https://github.com/bmurmann/ADC-survey

[R23] ngspice project. User manual and device-model documentation. Accessed 12 October 2026. https://ngspice.sourceforge.io/docs.html

[R24] SkyWater SKY130 PDK project. Device, corner, and process documentation. Accessed 12 October 2026. https://skywater-pdk.readthedocs.io/en/main/

[R25] University of California, Berkeley. EE105 MOS differential-pair lecture materials. https://rfic.eecs.berkeley.edu/courses/ee105/pdf/module4-6_Diffamps_p2.pdf

[R26] scikit-learn developers. Common pitfalls and recommended practices: preprocessing and data leakage. Accessed 12 October 2026. https://scikit-learn.org/stable/common_pitfalls.html

[R27] Banbury, C., et al. MLPerf Tiny Benchmark. NeurIPS Datasets and Benchmarks, 2021. https://arxiv.org/abs/2106.07597 ; current official benchmark: https://mlcommons.org/working-groups/benchmarks/tiny/

[R28] Rao, M., et al. Thousands of conductance levels in memristors integrated on CMOS. Nature 615, 823-829, 2023. https://www.nature.com/articles/s41586-023-05759-5

[R29] Demirkiran, C., Nair, L., Bunandar, D., and Joshi, A. A blueprint for precise and fault-tolerant analog neural networks. Nature Communications 15, 5098, 2024. https://www.nature.com/articles/s41467-024-49324-8

[R30] Chen, A., et al. Demonstration of transformer-based ALBERT model on a 14nm analog AI inference chip. Nature Communications 16, 8661, 2025. https://www.nature.com/articles/s41467-025-63794-4

Appendix A. Local evidence map and limits

The local inventory was read directly, rather than reconstructed from previous descriptions. Relevant refdesign files are xbar1t1r_device.py, xbar1t1r_netlist.py, xbar1t1r_train.py, digital_train.py, digital_netlist.py, digital_verify.py, verify_digital.log, trained exported decks, and final run/checkpoint artifacts. Export_verify.log reports that the three final crossbar exports match their saved final accuracies.

Relevant kfabric files are netlist_kfq.py, train_kfq.py, kfq_spec.py, netlist_kdn.py, gate_diff.py, gate_fq.py, simulator_api.py, ARCHITECTURE.md, and saved KFQ checkpoints/benchmark JSONs. Relevant ring files are netlist_ring.py, train_ring.py, dataset.py, and saved ring results. compare/tech.py, metrics.py, measure_existing.py, and verify_metrics.py define assumptions and metric helpers; compare's separate crossbar baseline is a different construction from refdesign's 1T1R baseline and should not be merged under one label.

The simulator source inspected was D:\Simulator\circuit_simulator with level3\src\models\mosfet_level1_model.py and its MOS component dispatcher. Saved code claims of numerical gradient agreement and circuit verification are useful context; they are not all independently rerun in this report. No new training, physical write-verify, PDK extraction, silicon energy, or timing measurement was performed.

The accompanying report builder saves an evidence JSON with source hashes, run summaries, and directly counted exported-deck components. This is a reproducibility record for the audit, not a substitute for immutable dataset and checkpoint metadata in future experiments.

Appendix B. Glossary

1T1R: one access transistor in series with one resistive storage device. MVM: matrix-vector multiplication. MAC: multiply-accumulate. TIA: transimpedance amplifier. DAC/ADC: digital-to-analog/analog-to-digital converter. ENOB: effective number of bits. QAT: quantization-aware training. STE: straight-through estimator. PVT: process, voltage, and temperature. PDK: process design kit. BPTT: backpropagation through time. KFQ: the local four-quadrant Kirchhoff network. KDN: the local two-transistor differential-column network. KRC: the recurrent-core design proposal. EDP: energy-delay product.

Common mode is the average of a differential pair; differential signal is its difference. A weight's common-mode conductance or gate voltage can affect power and operating headroom even when the ideal signed coefficient is unchanged. Software wall time, simulated physical time, hardware latency, and training time are four different quantities and must never be interchanged.